計算物理学B

第三回

野垣 康介、藤本 悠輝

講義予定

2026 年度 第 2 学期（火曜 3 限）

10/6	四則演算	12/8	モンテカルロ 1
10/27	制御文（for, if）	12/15	モンテカルロ 2
11/6（金）	**関数**	12/22	微分方程式 1
11/10	配列（numpy）	1/12	微分方程式 2
11/17	可視化（matplotlib）	1/19	偏微分方程式 1
11/24	数値微分	1/26	偏微分方程式 2
12/1	数値積分	2/2	根の探索
	～中間レポート～		～期末レポート～

10/20 は出張で休講。

11/6（金）は火曜授業の振替。

日付：令和 8 年度（2026）新潟大学授業暦

授業で使うリンク

- Google Colab
  https://colab.research.google.com/?hl=ja

- 講義資料（GitHub）
  https://github.com/nogaki/Computational_Physics_B

- 今週の教材
  - フォルダー：2026/week3
  - 実習用ノートブック：week3.ipynb

GitHub から Colab へ取り込む

1. GitHub で `week3.ipynb` を開き、
   ブラウザーのアドレス欄の URL をコピーします。

2. Colab で「ファイル」→「ノートブックを開く」
   →「GitHub」を選びます。

3. コピーした URL を貼り付けて検索し、
   表示されたノートブックを開きます。

開いたら「ドライブにコピーを保存」

「ファイル」メニューから、自分用のコピーを保存します。
開いただけでは、自分のドライブに保存されません。

関数：同じ処理の繰り返し

1 から 10 までと、1 から 30 までの和をそれぞれ求めます。

```
sum1 = 0
for i in range(10):
    sum1 = sum1 + (i + 1)

sum2 = 0
for j in range(30):
    sum2 = sum2 + (j + 1)

print(sum1 + sum2)
```

ほぼ同じ処理を、範囲だけ変えて 2 回書いています。

関数：処理をまとめる

```
def sum_one2n(n):
    total = 0
    for i in range(n):
        total = total + (i + 1)
    return total
sum1 = sum_one2n(10)
sum2 = sum_one2n(30)
print(sum1 + sum2)
```

出力 520

総和を求める処理を関数にまとめ、引数を変えて呼び出します。

関数の定義と返り値

```
def function_name(argument):
    return argument
```

- def の後に関数名と引数を書き、行末に: を付けます。
- 関数の中の処理は字下げします（空白４個を目安に）。
- 引数は複数指定できます。return で呼び出し元に値を返します。
- 返り値が複数の場合は、タプル（tuple）として返ります。
- 関数内で定義した変数は、関数の外では使えません。

print と return の違い

print は表示し、return は値を返します。return がない関数の返り値は None（値なし）です。

print で表示

```
def show(x):
    print(x * x)

a = show(3)
print(a)
```

出力
```
9
None
```

a には None が入ります。

return で値を返す

```
def square(x):
    return x * x

b = square(3)
print(b)
```

出力
```
9
```

b に入る 9 は後の計算に使えます。

関数内の変数

```
def sum_all(x, y, z):
    tmp = x + y
    return tmp + z

l = 3
m = 2
n = 5
print(sum_all(l, m, n))
```

出力 10

関数内で作った tmp は外から使えません。print(tmp) を実行すると、意図的に NameError になります。

引数と物理パラメータ

運動エネルギーは $E = \frac{1}{2}mv^2$ です。m は質量、v は速度です。
単位は、質量 m：kg、速度 v：m/s、エネルギー E：J です。

```
def kinetic_energy(m, v):
    return 0.5 * m * v ** 2

print(kinetic_energy(2.0, 3.0))
```

出力

```
9.0
```

呼び出しの１番目の値 2.0 が m、２番目の値 3.0 が v に入ります。

複数の返り値：自由落下

空気抵抗を無視し、静止状態から落下する物体を考えます。落下開始位置を $x = 0$、下向きを正とします。

計算例では重力加速度を $g = 10.0\ \mathrm{m/s^2}$ と近似します。経過時間 $t \geq 0$ は秒、位置 x はメートル、速度 v は $\mathrm{m/s}$ です。

$$x = \frac{1}{2} g t^2, \qquad v = g t$$

```
def free_fall(t):
    g = 10.0
    x = 0.5 * g * t ** 2
    v = g * t
    return x, v
```

複数の返り値：タプルを受け取る

カンマでまとめた値の組をタプルといいます。return x, v は、位置と速度をこの順でまとめて返します。

返した順で複数の変数に受け取ります。変数の数は返す値の数に合わせます。

```
state = free_fall(3.0)
print(state)
x, v = state
print(x, v)
```

出力

```
(45.0, 30.0)
45.0 30.0
```

x は $45.0\ \mathrm{m}$、v は $30.0\ \mathrm{m/s}$ です。

関数を組み合わせる

free_fallが返す速度をkinetic_energyの引数に渡し、運動エネルギーを計算します。

```
m = 2.0
x, v = free_fall(3.0)
energy = kinetic_energy(m, v)
print(energy)
```

出力

```
900.0
```

質量 $2.0\ \mathrm{kg}$、落下開始から $3.0\ \mathrm{s}$ 後の運動エネルギーは $900.0\ \mathrm{J}$ です。

計算結果を確かめる

実行前に、簡単な入力で結果を予想します。

```
print(kinetic_energy(2.0, 0.0))
print(free_fall(0.0))
```

出力 `0.0`

出力 `(0.0, 0.0)`

静止していれば運動エネルギーは 0 です。落下開始時 $t = 0$ の位置と速度も 0 です。

短い実習　基準を $m = 2.0\ \mathrm{kg},\ v = 3.0\ \mathrm{m/s}$ とします。質量だけを 2 倍、速度だけを 2 倍にした場合のエネルギーを別々に計算し、基準の何倍か説明しましょう。

関数の活用：漸化式

漸化式 $a_1 = 1, \quad a_{n+1} = 2a_n + 3$ の一般項は
$a_n = 4 \cdot 2^{n-1} - 3$ です。a_{10} を求めます。

```
def next_term(a):
    return 2 * a + 3

a = 1
for _ in range(9):
    a = next_term(a)

print(a)
```

出力　2045

最大公約数：ループ版

```
def gcd(n, m):
    while m != 0:
        r = n % m
        n = m
        m = r
    return n
print(gcd(120, 12))
```

ユークリッドの互除法：
正の整数 n, m に対し、
余り $r = n \bmod m$ を計算し、
(n, m) を (m, r) に更新。
$m = 0$ になるまで繰り返し、
最後の n が最大公約数です。

出力 12

実習タイム

例題　関数を使って次の値を求めましょう。

$$\sum_{n=1}^{10} a_n, \qquad a_n = n^2 + 4n + 5$$

1. **フィボナッチ数列**の第 10 項を、関数を使って求めましょう。
   $F_1 = F_2 = 1, \quad F_{n+2} = F_{n+1} + F_n$
2. 引数 n が素数かどうか判定する関数を作りましょう。

素数判定のヒント：`math.sqrt(n)` で平方根、`math.floor(x)` で $\lfloor x \rfloor$ を計算できます。

```
import math
```

発展：最大公約数（再帰版）

ループ版と同じ処理を、関数が自分自身を呼び出す再帰で書けます。

```
def gcd_recursive(n, m):
    if m == 0:
        return n
    return gcd_recursive(m, n % m)

print(gcd_recursive(120, 12))
```

出力 12

m == 0 が再帰の終了条件です。そうでなければ、次の組 (m, n % m) を計算します。